GUIDA DI ARCHITETTURA E INTEGRAZIONE

SmartPick_iC

Suite applicativa IEC 61131-3 per picking vision-guidato ad alta velocità: tracking su convogli, multi-pick fino a 4 pezzi, load sharing a quattro strategie, vassoi multi-livello, verifiche di processo e resilienza ai guasti. Architettura modulare con confine firmware isolato e framework di simulazione.

Target: Yaskawa iC 9226M-EC · iCube Engineer

Versione 1.2.0 · Ottobre 2026 · Libreria SmartPick (prefisso SP_)

SP-ARC-001 · DOCUMENTO TECNICO DI PROGETTO

Table of Contents

Executive Summary (Italiano) 2

1. System Overview & Feature Matrix 2

2. Architecture & Module Map 4

3. Data Model Catalog (DUTs) 6

4. Module Reference API 7

5. Firmware Integration Boundary 9

6. HMI Data Model & Wizard 10

7. Persistence & Recipe Format 11

8. Simulation Framework 12

9. Integration Guide (iCube Engineer) 12

9.1 Project setup . . . 12

9.2 Hardware mapping . . . 13

9.3 First commissioning checklist . . . 13

9.4 Binding the conveyor-tracking firmware FB . . . 13

9.5 Physical I/O map (HW-3) . . . 13

9.6 Camera presets and robot boot defaults (HW-4 / R-06) . . . 13

10. Test Plan 14

11. Constraint Compliance & Open Items 15

11.1 IEC 61131-3 / iCube constraint compliance . . . 15

11.2 Open items closure matrix . . . 16

12. v1.2 - Advanced Multi-Grip & Crash Prevention 17

12.1 Advanced multi-grip (R-07) . . . 17

12.2 Advanced crash prevention (R-05b) . . . 18

12.3 Professional layer (R-16) . . . 18

Executive Summary (Italiano)

SmartPick_iC è una libreria applicativa in linguaggio IEC 61131-3 Structured Text che replica e, dove indicato, supera le funzionalità di Yaskawa MotoPick 4 su piattaforma iCube Engineer, con target hardware iC 9226M-EC. La suite gestisce applicazioni di picking guidato dalla visione su convogli in movimento: tracking dei pezzi con encoder, riservazioni cut-off alla linea di prelievo, multi-pick e multi-place asincrono fino a 4 pezzi per ciclo, distribuzione del carico tra robot con quattro strategie configurabili, generazione di pattern di piazzamento con vassoi multi-livello, gestione batch, verifiche di presa e deposizione, slow/stop automatico dei convogli, pausa/ripristino senza perdita dei pezzi tracciati e resilienza ai guasti robot con trasferimento automatico del controllo camere.

Il progetto è organizzato in 15 moduli (66 sorgenti, oltre 12.000 righe di ST, oltre 1.100 dichiarazioni di variabile ciascuna corredata da un commento esplicativo verificato con scanner dedicato) con un confine firmware ristretto e documentato: tutte le chiamate al firmware (gruppi cinematici, assi encoder, socket, file) sono isolate in cinque FB adapter recanti marcati CONFIRM. Il resto della libreria è portabile e testabile offline grazie al framework di simulazione integrato (convogli e telecamere virtuali, cinematica simulata). I vincoli di progetto richiesti sono rispettati in modo sistematico: nessun puntatore, nessun reference, nessuna ACTION, nessun array di FB, nessuna ricorsione, metodi con sole VAR/VAR_IN, chiamate ai metodi sempre tramite THIS, VAR_IN_OUT solo sugli adapter e sempre assegnati, costanti in GVL per non violare la regola DDT di iCube Engineer.

La versione 1.2 aggiunge tre famiglie di funzioni professionali. La gestione multi-grip avanzata (R-07) forma cluster geometrici di più pezzi compatibili con la geometria della pinza e li preleva con una singola discesa sincronizzata, con verifica del vuoto per singola coppa, re-grip individuale delle coppe fallite, controllo di payload e baricentro e deposizione sincronizzata con fallback automatico al prelievo/adeposizione sequenziale. La crash prevention avanzata (R-05b) estende l'interlock di zona con acquisizione ordinata di più zone a prova di deadlock, limiti di velocità per zona, separazione dinamica TCP- TCP tra robot con rallentamento graduale e stop latching, priorità di servizio e diagnostica di contesa. Il livello professionale (R-16) completa il quadro con OEE/MTBF/MTTR, contatori di manutenzione predittiva e suite di self-test di avvio.

Il pacchetto consegnato comprende: i sorgenti ST organizzati per modulo con ogni variabile commentata, due ricette CSV iniziali (Demo_Sim per la simulazione e Demo_Line1 come template di impianto reale) con validatore offline e riferimento campi bilingue, gli allegati di integrazione (patch firmware di tracking, mappa I/O di impianto, matrice di chiusura dei punti aperti) e questa guida di architettura e integrazione con piano di test. I punti di conferma residui per la messa in servizio sono riepilogati nel capitolo 11.

> **Scala supportata**: 4 robot (gruppi cinematici firmware) - 8 convogli/feed - 2 telecamere - 4 tool per robot - 32 tipi prodotto - 64 pezzi tracciati per feed - fino a 4 pezzi per ciclo di prelievo. I limiti sono costanti GVL modificabili.

1. System Overview & Feature Matrix

SmartPick_iC addresses the classic End-Of-Line packaging problem: picking unsorted products from moving infeed conveyors and placing them into moving or static targets (trays, cases, outfeed belts). The product types can be mixed on the same belt; the library identifies each item by product type and matches it against user-defined placement batches. Kinematics are not handled by the library: delta, SCARA, gantry and H-bot mechanisms are firmware kinematic groups on the iC 9226M-EC, and SmartPick commands them through a narrow, well-marked adapter boundary. This mirrors the MotoPick 4 philosophy of selling systems rather than robots, while keeping every feature accessible from standard IEC 61131-3 code.

The table below summarizes the functional parity between SmartPick_iC and the MotoPick 4 / MotoPick_iCube feature set described in the Yaskawa material. Every listed feature is implemented in the delivered sources; the ones marked as firmware-assisted rely on the conveyor-tracking firmware option for full-speed operation and fall back to a cut-off predicted pose when the option is not bound yet.

Feature	MotoPick 4	SmartPick_iC	Where
Vision-based picking from moving belts	Yes	Yes	SP_VisionCam + SP_TrkFeed
Tracking speed up to 1 m/s	Yes	Yes (FW tracking option / cut-off)	SP_RobMotion
Pause & resume without losing items	Yes	Yes (encoder-based continuity)	Queues + FSM Paused
Async multi-pick/place up to 4 parts	Yes	Yes (GC_CUPS = 4, Sync/Async)	SP_Robot multi-grip
Different part types picked & sorted	Yes	Yes (type-matched targets)	SP_Dispatch + queues
Load balancing strategies	organic/greedy/balanced/ manual	Same 4 + match modes	SP_LoadBalancer
User share ratios & work ranges	Yes	Yes (per robot x feed)	ST_SP_ShareCfg
Multi-conveyor per camera	Yes	Yes (region table, speed-scaled triggers)	SP_VisionCam
Misalignment compensation via calibration	Yes	Yes (N-point similarity fit)	SP_Calib2D
Slow & stop conveyor control	Yes	Yes (per-feed thresholds, auto restart)	SP_Conveyor
Multi-layer tray pick/place	Yes	Yes (up to 4 layers)	SP_PatternGen
Pick/place order & batches	Yes	Yes (priority, type mix)	SP_BatchMgr
3-point motion, offsets, precision modes	Yes	Yes (per robot x feed x type)	ST_SP_PathOff
Processing verification + cycle restart	Yes	Yes (pick/place/deferred + retries)	SP_Robot FSM
Wizard on HMI, online adjustment	Yes	Data model + config apply provided	GVL_SP_Hmi

Feature	MotoPick 4	SmartPick_iC	Where
Comprehensive logging	Yes	Yes (ring buffer + file dump)	SP_Logger
Failure resiliency (robot loss, camera handover)	Yes	Yes	SP_System
Crash prevention between robots	No (listed as missing)	Yes: zone interlock + speed limits + dynamic TCP separation	SP_ZoneGuard + SP_Collision

Table 1-1 - Feature matrix versus MotoPick 4 (DS-984) and the MotoPick_iCube deck.

System limits are compile-time constants in GVL_SP_Const. The delivered values fit the reduced iC 9226M-EC configuration selected for this project; they can be raised when the application requires the full MotoPick 4 scale and the controller memory allows it. Raising GC_ROBOTS or GC_FEEDS requires adding the corresponding explicit FB instances in PRG_SP_Main, because arrays of function blocks are not used anywhere in the library.

Constant	Value	Meaning
GC_ROBOTS	4	Robots / kinematic groups
GC_FEEDS	8	Tracked conveyors (feeds), incl. 2 outfeeds
GC_CAMS	2	Cameras (each may serve multiple feeds)
GC_TOOLS / GC_CUPS	4 / 4	Tools per robot / cups per tool
GC_TYPES	32	Product types
GC_QUEUE	64	Tracked items per feed
GC_LAYERS	4	Tray layers per pattern
GC_PATT / GC_BATCHES	8 / 8	Patterns / batch definitions
GC_LOG_N / GC_ALARM_N	256 / 64	Log ring / alarm registry depth

Table 1-2 - Compile-time limits (GVL_SP_Const).

2. Architecture & Module Map

The library is layered so that the picking domain logic never touches the firmware directly. Five adapter function blocks form the only integration surface: SP_RobMotion (kinematic group motion), SP_ConvEncoder (belt position acquisition), SP_ConvDrive (belt speed command), SP_Socket (TCP for cameras) and SP_FileIo (file system for recipes and log dumps). Each adapter contains CONFIRM comments describing exactly which firmware call must be verified on the target; replacing or adjusting a binding never touches the domain layer. Because references and FB arrays are forbidden, the wiring program PRG_SP_Main owns every instance explicitly and shuttles data through public mirror structures; SP_System and SP_Dispatch read the same mirrors and publish their decisions as plain data that the wiring program

applies back to the FB instances.

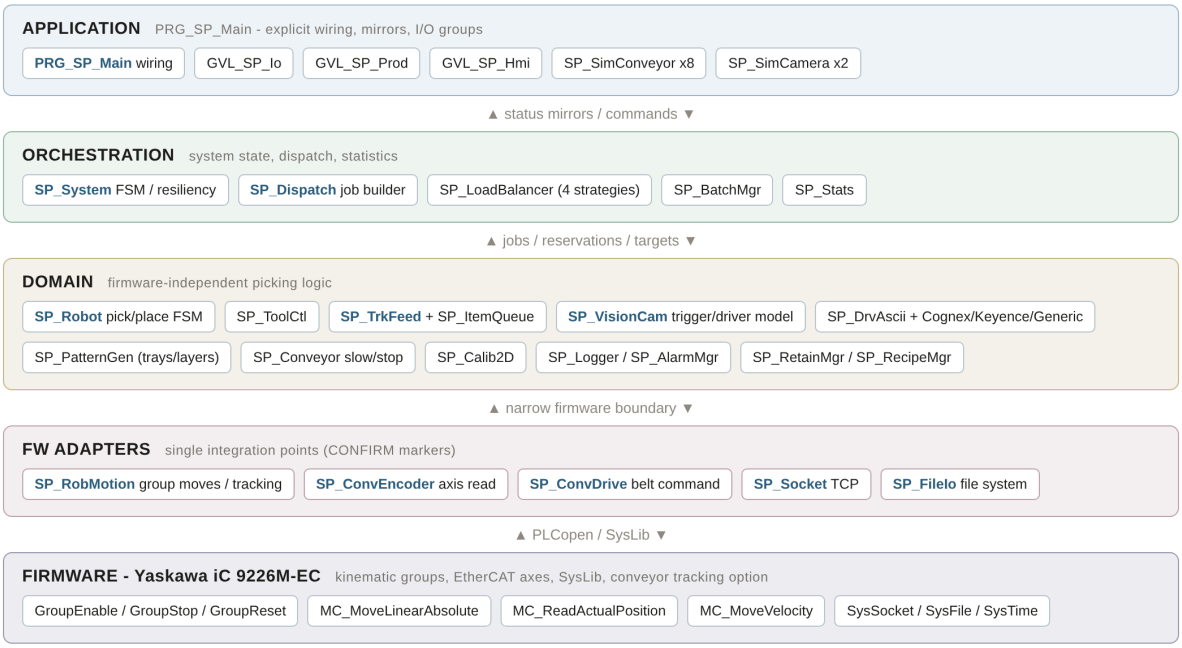


Figure 2-1 - Layered architecture: domain logic is firmware-independent; all firmware calls live in the five adapter FBs.

Module	Contents	Responsibility
01_Common	DUT enums/structs, GVL_SP_Const, utils, parser, clock, MovAvg, Logger, AlarmMgr	Shared foundations
02_Geometry	Point/Pose/Transf2D, trig helpers, SP_Calib2D	Frames and calibration
03_Conveyor	ConvEncoder, ConvDrive, Conveyor	Belt acquisition, slow/stop, drive
04_Vision	Socket, DrvAscii, Cognex/Keyence/Generic, VisionCam	Trigger policies, parsing, routing
05_Tracking	ItemQueue, TrkFeed	Item flow, cut-off reservation
06_Pattern	PatternGen, BatchMgr	Targets, trays/layers, batches
07_LoadSharing	LoadBalancer, ZoneGuard	Distribution strategies, zone interlock (R-05)
08_Robot	ToolCtl, RobMotion, Robot	Pick/place execution
09_Production	System, Dispatch, Stats	Orchestration, job building, KPI
10_Persistence	FileIo, RetainMgr, RecipeMgr	Retain, recipes, log dump
11_Simulation	SimConveyor, SimCamera	Offline test harness
12_Application	GVLs (Io/Prod/Hmi/Fw), RobCfgDefault, PRG_SP_Main	Wiring, mirrors, HMI hub, FW status, boot defaults
13_MultiGrip	Cluster DUTs, ClusterMath, GripPlanner	Advanced multi-grip cluster intelligence (R-07)

Module	Contents	Responsibility
14_Collision	ColTypes, Collision	Advanced crash prevention: zones, speed, separation (R-05b)
15_Professional	ProTypes, Oee, Maint, SelfTest	OEE/MTBF/MTTR, predictive maintenance, self-test (R-16)

Table 2-1 - Module map (one folder per module in the delivered package).

The cyclic data flow in PRG_SP_Main follows a strict order that guarantees the single-writer property of every mirror: time source, simulated belts, real conveyors, feed housekeeping and mirrors, robot bodies and mirrors, cameras and item drain into feed queues, pattern engines and target insertion, dispatcher handshake, system orchestrator, and finally services (logger, alarms, retain, recipe, HMI mirrors). This ordering means the dispatcher always sees the state after motion and vision updates of the same scan, and robots receive jobs whose queue records are still consistent.

3. Data Model Catalog (DUTs)

Types follow strict prefixes: ET_ for enumerations, ST_ for structures, GC_ for constants. Enumerated values are always referenced with the DUT-qualified form ET_Xxx#Member, which respects the iCube Engineer rule that constants declared in a data type worksheet are usable only inside that worksheet. Compile-time limits live in the GVL worksheet GVL_SP_Const as VAR CONSTANT, not in a DDT, so every worksheet can use them. Strings have fixed lengths everywhere and all records carry an explicit validity flag instead of sentinel values.

Group	Types	Notes
Common	ET_SP_RetCode, ET_SP_LogLevel, ET_SP_LogSource, ET_SP_SysState, ET_SP_AckState, ET_SP_ItemState, ET_SP_RobState, ET_SP_Precision, ET_SP_MatchMode, ET_SP_ShareStrategy, ET_SP_GripMode, ET_SP_JobKind, ET_SP_BatchState, ET_SP_PattTrig, ET_SP_VisionDrv, ET_SP_VisionState, ET_SP_TrigMode, ET_SP_ConvDriveMode, ET_SP_SocketState, ET_SP_FileState; ST_SP_Version, ST_SP_LogEntry, ST_SP_Alarm, ST_SP_Counters	Shared vocabulary of the whole library
Geometry	ST_SP_Point2D/3D, ST_SP_Pose2D/3D, ST_SP_Transf2D, ST_SP_CalibStat	Similarity transform: parent = T o child
Conveyor	ST_SP_ConvCfg, ST_SP_ConvStat	Scaling, direction, window, slow/stop thresholds
Vision	ST_SP_VisionItem, ST_SP_CamFeedMap, ST_SP_VisionCfg, ST_SP_VisionStat	Item record, region table, driver/protocol config
Tracking	ST_SP_TrkItem, ST_SP_ResReq, ST_SP_ResRsp, ST_SP_TrkCfg, ST_SP_TrkStat	Tracked record with capture stamps and prediction
Pattern/Batch	ST_SP_PattDef, ST_SP_BatchDef, ST_SP_BatchStat	Grids, layer stack, type mix, quantity

Group	Types	Notes
Load sharing	ST_SP_RobShare, ST_SP_Offer, ST_SP_ShareCfg	Balancer I/O and configuration
Robot	ET_SP_MvKind, ST_SP_ToolCfg/Stat, ST_SP_PathOff, ST_SP_RobCfg, ST_SP_JobItem, ST_SP_Job, ST_SP_RobCmd, ST_SP_RobStat	Tools, 3-point offsets, jobs, status
Production	ST_SP_FeedIo, ST_SP_SysCmd, ST_SP_SysCfg, ST_SP_SysStat, ST_SP_DispReq, ST_SP_CancelReq	Mirrors and dispatch records
Persistence	ST_SP_RetainData, ST_SP_RecipeHdr	Retain blob and recipe header
MultiGrip (13)	ET_SP_GripExec, ST_SP_ClReq, ST_SP_ClRsp	Cluster request/response, synchronized execution class
Collision (14)	ET_SP_ColAct, ST_SP_ColCfg, ST_SP_ZoneDiag, ST_SP_RobCol	Guard configuration, per-zone and per-robot verdicts
Professional (15)	ST_SP_OeeView, ST_SP_MaintCfg, ST_SP_MaintStat, ST_SP_TestRes	OEE snapshot, maintenance thresholds/counters, self-test result

Table 3-1 - DUT catalog (file names DUT_SP_*.st).

The two central records are ST_SP_VisionItem, produced by the camera layer, and ST_SP_TrkItem, the tracking record enriched with capture stamps, world pose, reservation owner and target metadata. ST_SP_TrkItem.rZWorld carries the working height: belt surface for products, layer Z for placement targets. The reservation response ST_SP_ResRsp already contains the predicted pick pose on the pick line, so the robot layer never repeats frame mathematics.

4. Module Reference API

Every public method uses only VAR and VAR_INPUT parameters and returns a scalar or an enumeration; complex results are published as public output members (prefix out_) that callers read after the call. Method calls from inside a parent FB always go through THIS.Method(). FB bodies execute the state machines and all firmware-bound calls; methods implement pure decisions, which keeps them free of the VAR_IN_OUT restriction and easy to unit-test. The tables below list the essential API per module; full signatures are documented in the source headers.

FB	Key methods (VAR/VAR_IN only)	Purpose
SP_Conveyor	Configure, FactorFor, InWindow	Speed filter, slow/stop arbitration (auto restart)
SP_VisionCam	Configure, TriggerNow, GetNextItem, InjectItem	Trigger policy engine, item routing, sim path
SP_DrvAscii / brand drivers	Configure, GetItem, ClearItems	Trigger, parse, sequential item access
SP_Calib2D	Reset, AddPair, Compute	N-point similarity calibration with RMS

FB	Key methods (VAR/VAR_IN only)	Purpose
SP_ItemQueue	Configure, Add, Reserve, ReserveCluster, CompletePick, Release, NearDist, Housekeeping, AgeItems	Ring buffer of tracked items + cluster reservation (R-07)
SP_TrkFeed	Configure, FeedItem, AddTarget, Reserve, ReserveCluster, CompletePick, PlaceTarget, NearDist, Housekeeping	Frame transforms + queue management
SP_PatternGen	Configure, StartBatch, AbortBatch, Engine, OnSensor, NotifyPlaced, GetNextTarget	Pattern/tray/layer target generation
SP_BatchMgr	AddBatch, StartBatch, StopBatch, GetActive, GetDef, GetStat, SyncProgress	Batch registry and priority arbitration
SP_LoadBalancer	Configure, OfferProduct	4 strategies + match modes (combinational)
SP_ZoneGuard	Configure, body-driven: abRobReq/aiRobFeed -> abGrant/abHold	One-latch-per-zone interlock, transparent when disabled (R-05)
SP_Collision	Configure + body: aiReq/ap2Pos -> astRob (grant, scale, reason), astZd	Advanced guard: ordered multi-zone, speed limits, TCP separation (R-05b)
SP_GripPlanner	Begin, OfferCand, Finish (pull API from the queue)	Cluster formation: cup matching, pruning, payload/CoG audit (R-07)
SP_ToolCtl	Configure, GripStart, GripSync, PlaceStart, FbkCup, AbortAll, HoldingOf, ResetCounters	Tool signals, sync grip with per-cup capture, dry-run inhibit
SP_Robot	Configure, OfferJob, CancelJob, MarkFault, ClearFault; rVelScale input	Pick/place FSM incl. synchronized cluster descent + re-grip
SP_System	(body Step) public: Bal, Batch, Stats, stCmd, enable vectors	Startup staging, resiliency, batch service
SP_Dispatch	(body Run) public: stShare, aiOutfeed, handshake arrays	Per-robot job building via balancer
SP_Logger / SP_AlarmMgr	Log/LogInfo/LogWarn/LogError/GetEntry/ExportLine; Raise/Clear/Ack/AckAll/GetActive	Diagnostics
SP_RetainMgr / SP_RecipeMgr	Validate/RestoreDefaults/SaveNow/SetRecipe; ExportBegin/ExportNext/ImportLine	Persistence
SP_Oee	body + Reset(stCnt), MTReset	OEE = A x P x Q, time budget, MTBF/MTTR (R-16)
SP_Maint	body + ResetAll (thresholds in stCfg)	Grip/camera/belt/pump wear counters with due flags (R-16)
SP_SelfTest	bRun one-shot -> stRes (ST_SP_TestRes)	7 configuration consistency checks at startup (R-16)

Table 4-1 - Public API surface (methods are documented in FB file headers).

The robot job protocol is the core integration contract. SP_System (through SP_Dispatch) builds an ST_SP_Job containing up to GC_CUPS items, each with the reserved source item, the predicted pick pose, the target pose and the tool/cup assignment. The job is offered with OfferJob; the robot plans the

approach/pick/depart and place sequence per item, executes grip and release with verification, and finally pulses bJobDone while publishing m_stJobRes with per-item flags. The wiring program then closes the queue records (CompletePick or PlaceTarget), notifies the pattern generator about placed slots and feeds the statistics. Refused offers and mid-handshake cancellations are released through a dedicated cancel record so that no reservation can leak.

5. Firmware Integration Boundary

Five adapters contain every firmware interaction. Each call site carries a CONFIRM comment; the table lists the assumed binding and what must be verified on the installed iCube Engineer library set. Bindings are deliberately standard PLCopen names where possible, because those are the most stable part of the Yaskawa motion API; the only genuinely target-specific element is the conveyor-tracking function block, which has a single dedicated call site (CONFIRM-TRACKING in SP_RobMotion). Until that binding is done, tracking moves use the cut-off predicted pose with the stock PLCopen linear move, which is functional at moderate belt speeds and is the configuration exercised by the simulation framework.

Adapter	Firmware call (assumed)	To verify on target
SP_ConvEncoder	MC_ReadActualPosition(Axis)	Works on encoder axis; scaling source
SP_ConvDrive	MC_MoveVelocity; analog via GVL_SP_Io.grConvDrive	Mode enum, %QW scaling of analog channels
SP_RobMotion	GroupEnable/GroupStop/GroupReset; MC_MoveLinearAbsolute; group pose read	Group Ready output; pose output mapping; CONFIRM-TRACKING: replace linear move with the Yaskawa tracking FB for full-speed tracking
SP_Socket	SysSocketOpen/Send/Receive/Close	Signatures and handle type; timeout behavior
SP_FileIo	SysFileOpen/Read/Write/Close	Mode enum values; directory conventions
PRG_SP_Main sec.1	Controller clock read	Service name (e.g. SysTimeGetDT)

Table 5-1 - FW binding table. Search for CONFIRM in the sources.

Pointer policy: the SysSocket and SysFile APIs require address arguments, so ADR() appears only inside SP_Socket and SP_FileIo, applied to internal buffers, and never in user data types or domain code. No POINTER TO or REFERENCE TO is declared anywhere in the library. Byte-to-character conversion for socket traffic uses the printable-ASCII lookup table GC_ASCII95 through FUN_SP_ByteCh, keeping the receive path free of pointer arithmetic. If the plant policy forbids ADR() entirely, the two adapters are the only files to revisit, and the camera protocol can be moved to a fieldbus-based driver without touching the domain layer.

> **Open item FW-1 (blocking for full-speed tracking)**: provide the exact POU names and signatures of the iCube conveyor-tracking FBs for iC 9226M-EC. The integration point is the call marked CONFIRM-TRACKING in SP_RobMotion; the guide section 9.4 describes the expected call pattern.

Group status (FW-2, closed in code): the firmware status of each kinematic group is decoupled from the domain logic through GVL_SP_Fw. Two operating modes are provided. Proxy mode (default) preserves the original behavior, where SP_RobMotion aliases the homed condition to the enable state. Inject mode lets the wiring program publish the real group Ready and referenced bits read from the firmware status structure into g_abFwGrpReady / g_abFwGrpRef; PRG_SP_Main routes them into the robots as bFwReadyIn / bFwRefIn, and SP_Robot exposes them as bFwReady / bFwRef. The step-by-step identification procedure for the tracking FBs (library repository search, parameter mapping, drop-in variants A and B, five-test commissioning plan) is documented in Firmware/FW_PATCH_ConveyorTracking_IT-EN.md, delivered with the package in bilingual form.

6. HMI Data Model & Wizard

The HMI binds exclusively to GVL_SP_Hmi and GVL_SP_Prod; all logic state remains inside the function blocks. Commands are one-shot flags that the wiring program latches and the logic consumes, which makes the interface safe for any HMI generator. The wizard flow mirrors the commissioning sequence: system options, conveyor setup, camera configuration and calibration, placement patterns, batch definitions, load-sharing setup, and tool configuration. Each wizard page edits the corresponding configuration mirror; the Apply command propagates the mirrors into the function blocks through their Configure methods, which validate every field and reject inconsistent sets with explicit return codes.

Wizard page	Data area	Apply path
System options	g_stSysCfg	SP_System.stSysCfg
Conveyors	g_astConvCfg[1..8]	SP_Conveyor.Configure (validation)
Feeds / frames	g_astFeedCfg[1..8]	SP_TrkFeed.Configure
Cameras + calibration	g_astCamCfg[1..2], SP_Calib2D	SP_VisionCam.Configure + transforms into feed cfg
Placement patterns	g_astPatt[1..8]	Read by SP_PatternGen at batch start
Batches	g_astBatchDef[1..8]	SP_BatchMgr.AddBatch
Load sharing	g_stShareCfg	SP_LoadBalancer.Configure, SP_Dispatch.stShare
Robots & tools	g_astRobCfg[1..4]	SP_Robot.Configure
Operation	g_hmiCmd	SP_System.stCmd via PRG_SP_Main
Diagnostics	g_astHmiAlarm/Log, stats	Cyclic mirrors

Table 6-1 - Wizard pages and their data paths.

Online adjustment during production is intentionally limited to safe parameters: speed factors, share ratios, trigger distances and verification retries take effect at the next cycle. Structural configuration (scale limits, frame transforms, tool geometry) requires the system to be stopped, matching the single-writer contract of the configuration mirrors. Every configuration change is logged with source Persist, giving traceability during commissioning.

7. Persistence & Recipe Format

Persistence has two layers. The retain blob (VAR RETAIN in GVL_SP_Prod, managed by SP_RetainMgr) keeps power-failure continuity: lifetime counters, last recipe name, camera control ownership and batch progress. A checksum over the payload distinguishes a valid restore from first boot; on mismatch the manager restores documented defaults and flags the HMI. Recipes are product configurations and use the file system through SP_FileIo in fixed-record CSV: one line per object, comma separated, positional fields. The positional format was chosen over key=value INI because the same tokenizer (FUN_SP_TokN) used by the vision drivers parses it, and because each record maps to exactly one configuration structure with no ambiguity.

Record	Tokens (after the record name)	Target
CONV,n,...	scale, dir, maxspd, winstart, winlen, slowahead, stopahead, slowf, drive, anach	g_astConvCfg[n]
FEED,n,...	dir, winstart, winlen, pickahead, timeout_ms, zbelt, worldDx, worldDy, worldRot	g_astFeedCfg[n]
CAM,n,...	drv, ip, port, trig, trigdist, refspd, maxitems, thdeg, minscore, delay_ms	g_astCamCfg[n]
CAMMAP,n,k,...	feed, ymin, ymax, trigd, refspd	camera region table
PATT,n,...	rows, cols, dx, dy, rotstep, bx, by, bth, layers, layerh, zbase, trigdist	g_astPatt[n]
BATCH,n,... / BATCHMIX,n,k,...	outfeed, patt, qty, trig, priority / type, cnt	g_astBatchDef[n]
SHARE,f,... / SHARER,f,r,...	strat, match, strict, pickbias, outfeedmap / ratio, rangef, ranget, tmask, manual	g_stShareCfg

Table 7-1 - Recipe record layout (v1). Header line: #SmartPick_iC Recipe v1,name.

Import runs only while production is stopped (single-writer contract) and finishes by re-applying the configuration to every FB through the bApplyConfig path. Robot and tool commissioning data (velocities, grip times, cup geometry) is deliberately excluded from the product recipe and lives in the commissioning set; this separation keeps recipes machine-independent. The log ring can be dumped to file on demand from the HMI with one line per cycle, so even large dumps never block the task.

Two ready-to-use recipe files ship with the package. Demo_Sim.csv matches the simulation wiring of PRG_SP_Main (8 conveyors, 2 simulated cameras, feeds 1-6 as infeeds and 7-8 as outfeeds via patt1/patt2,

Greedy strategy with Mixed fallback, full 32-bit type masks, work zones A = robots 1-2 / B = robots 3-4) and is the fastest path to a successful g_bSim = TRUE dry run. Demo_Line1.csv is the real-cell template (2 infeeds + 2 outfeeds, CAM1 = Cognex on port 5000, CAM2 = Keyence on port 8888) whose IP addresses, ports and calibration values must be adapted on site. The offline validator Recipes/SP_RecipeCheck.py replays the same rules implemented by ImportLine (token counts, ranges, cross-references batch-to-pattern-to-outfeed, camera map references, ASCII/BOM hazards) and both delivered files PASS; Recipes/README_Recipe_IT-EN.md documents every field bilingually together with the load procedure and the merge semantics (an absent record leaves the existing configuration unchanged). Fields that exceed the 16-bit range (type masks, batch quantities, ms timeouts and delays) are parsed by the 32-bit FUN_SP_StrToUd, so the full UDINT domain round-trips through export and import.

8. Simulation Framework

The simulation framework makes the whole application runnable on the engineering laptop before any hardware exists. SP_SimConveyor integrates the commanded speed into a monotonic belt position; SP_Conveyor consumes it in bSim mode instead of reading an encoder axis. SP_SimCamera spawns deterministic pseudo-random item batches (32-bit LCG seed) along the belt and injects them into SP_VisionCam through InjectItem, so the complete downstream path - frame transforms, queues, cut-off reservation, load sharing, job building, robot state machines, verification and statistics - runs exactly as in production. Robot kinematics are emulated inside SP_RobMotion (bSim = TRUE): poses interpolate toward targets at the commanded velocity and motion completion pulses behave like the firmware ones.

- **Scenario controls**: g_bSim gate in GVL_SP_Prod; per-camera spawn distance, items per image, type mix and span; external spawn edges for scripted tests.
- **What to expect**: items flow from camera to queue, dispatcher builds jobs, robots cycle pick/place, batches complete, counters and pieces-per-hour converge.
- **What is not simulated**: physical cycle-time realism of the kinematics, crash avoidance, conveyor slip; those require hardware validation.

The recommended bring-up sequence is: run the simulation with one robot and one feed until counters move; add the second camera and verify region routing; enable all four robots and watch the load-sharing strategies distribute the flow; finally inject a robot fault (MarkFault) and verify camera ownership transfer and dispatch exclusion. The test plan in chapter 10 references exactly these scenarios.

9. Integration Guide (iCube Engineer)

9.1 Project setup

Create the worksheets in the import order documented in 00_ImportOrder.st: data types first, then global lists, functions, function blocks with their methods, and finally the program. Method worksheets are created under their parent FB with the exact names found in the METHOD banners of each FB file. If the tool enforces one TYPE per DUT worksheet, split the multi-type files keeping the exact names. Assign

PRG_SP_Main to the motion task (CONFIRM the task period against g_uiDtMs in GVL_SP_Prod).

9.2 Hardware mapping

Bind g_aAxisEnc[1..8] to the conveyor encoder axes (or belt servo axes) and g_aGrp[1..4] to the kinematic groups configured in the hardware catalog. Map the tool signal arrays of GVL_SP_Io to the physical terminals following the automatic channel convention (robot-1)*GC_TOOLS+tool, or set the per-tool channel overrides in ST_SP_ToolCfg. Analog conveyor drives use grConvDrive[i] mapped to a scaled %QW channel; servo belt axes use the MC_MoveVelocity path of SP_ConvDrive instead.

9.3 First commissioning checklist

- Run with g_bSim = TRUE and verify the full logic cycle offline (chapter 8).
- Resolve every CONFIRM marker: encoder read, drive mode, group binding, clock read.
- Home the groups at machine level, then verify SP_RobMotion.bHomed.
- Calibrate each camera against its feed with SP_Calib2D (2 points minimum, RMS check).
- Set the conveyor window, slow/stop thresholds and the pick line; validate with one item.
- Enable the conveyor-tracking firmware binding (CONFIRM-TRACKING) before speeds above 0.5 m/s.
- Dry-run (trial run) the first jobs with tool outputs inhibited.
- Switch g_bSim to FALSE only after the verification checklist of chapter 10 passes.

9.4 Binding the conveyor-tracking firmware FB

The tracking call site is isolated in SP_RobMotion, marked CONFIRM-TRACKING. The expected pattern for the Yaskawa tracking FB is: enable the tracking relationship between the robot group and the feed encoder once at startup, then issue the tracked move with the target expressed in conveyor coordinates (the item world pose plus the belt offset at reservation), letting the firmware maintain synchronization until the grip command. The adapter already publishes everything the tracking FB needs: ST_SP_ResRsp carries the predicted pose and the belt speed; the reservation was taken at the cut-off point configured per feed. After binding, re-run the tracking test cases (TC-05, TC-06) at production speed.

9.5 Physical I/O map (HW-3)

Hardware/HW_IO_Map_IT-EN.md contains the terminal-level template of the whole cell: safety chain and E-stop integration, 16+16 tool channels following the (robot-1)*GC_TOOLS+tool convention, the 8 conveyor drives in analog or servo-axis variant, 8 tray sensors, 8 encoder axes, the 4 kinematic groups, camera trigger lines and the 4 ms motion task layout, closed by a 9-step commissioning checklist. Fill the iCube hardware view following that document, then bind the arrays of GVL_SP_Io and the axis / group references of GVL_SP_Prod; no domain worksheet requires changes, because the map only feeds existing mirror variables.

9.6 Camera presets and robot boot defaults (HW-4 / R-06)

FUN_SP_CamPreset applies per-model-family defaults to any camera configuration field still at its neutral value, leaving explicitly configured values untouched. The model selector is GVL_SP_Hmi.g_aiCamModel per camera (-1 = Cognex preset, 0 = keep current, 1 = Keyence, 2 = generic TCP, 3 = simulated) with documented port defaults (Cognex ASCII job 5000, Keyence ASCII 8888). The layer composes with the driver protocol presets applied afterwards by the brand FBs. On site, confirm each camera job output format against the CONFIRM tags in the function header. FUN_SP_RobCfgDefault solves the first-boot problem for robot commissioning data: before SP_Robot.Configure, PRG_SP_Main applies a one-shot generic suction profile (1 tool, 2 cups, 2000/1200/1800 mm/s speeds, 2 s grip timeout, one verification retry, Z offsets +80/+40 mm) that is idempotent and fully overridable from the HMI wizard.

10. Test Plan

Test cases are numbered per module and reference the simulation framework where hardware is not required. Each case lists precondition, steps and expected result; execution logs go to the SmartPick logger for traceability. The set below is the minimum acceptance gate before on-machine commissioning.

ID	Scope	Steps	Expected
TC-01	ItemQueue FIFO/offsets	Feed 10 items, advance encoder, read offsets	Offsets match encoder delta x direction; purge behind pick line
TC-02	Cut-off reservation	Two robots, overlapping ranges, items at different offsets	Nearest item reserved first; second robot gets next; no double reservation
TC-03	Retry policy	Force pick NG (no feedback), retries = 2	Item re-tracked twice, then Lost; counters pickNg + 1
TC-04	Trigger distance	Belt at 2 speeds, camera on 2 feeds	Trigger intervals scale with speed ratio within 0.5..2.0
TC-05	Tracking accuracy (FW)	Bound FW tracking FB, belt 0.5-1.0 m/s	Pick within tolerance window; no missed items at max rate
TC-06	Cut-off fallback	Stock linear move, belt 0.3 m/s	Flying pick succeeds with tuned pickAhead
TC-07	Slow/stop	Items stacked toward pick line	Belt slows at rSlowAhead, stops at rStopAhead, auto restarts
TC-08	Load sharing	4 strategies x 4 robots, mixed types	Distribution matches strategy and ratios; exclusions honored
TC-09	Multi-grip	uiMaxGrip = 4, Sync and Async modes	Up to 4 picks per cycle; cup poses correct; tool single-type enforced
TC-10	Layer/tray	3x2 pattern, 2 layers, sensor trigger	Layer Z advances after layer complete; new tray on sensor edge
TC-11	Batch completion	Qty = 24, type mix 2 entries	Batch Done at quantity; priority arbitration on shared outfeed

ID	Scope	Steps	Expected
TC-12	Pause/resume	Pause during multi-pick job	Job frozen, reservations kept, resume completes cycle; belt items still tracked
TC-13	Verification & restart	DeferVerify with part lost at place	Place NG flagged, retry, then alarm; cycle restart per config
TC-14	Recipe round-trip	Export, modify, import, apply	All mirrors equal; Configure returns Done; production stopped during import
TC-15	Power-failure continuity	Pick counters, load recipe, power cycle (retain)	Retain restored (checksum), counters and batch progress kept
TC-16	Robot loss resiliency	MarkFault on one robot under load	Dispatch excludes it, camera ownership transfers, production continues
TC-17	Alarm lifecycle	Raise, acknowledge, clear	States Active/Acked/Inactive; occurrence counter increments
TC-18	Log dump	Export 64 entries to file	File contains complete ASCII lines; task not blocked

Table 10-1 - Acceptance test cases (TC-05/06 require hardware).

System acceptance additionally requires a two-hour production run in simulation with all modules active, verifying: no queue overflow alarms under the configured spawn rate, pieces-per-hour stability within the expected band, and zero reservation leaks (items in state Reserved without an owning robot at idle). On machine, repeat TC-05 through TC-07 at production speed before releasing to operations.

11. Constraint Compliance & Open Items

11.1 IEC 61131-3 / iCube constraint compliance

Constraint	Status	Evidence
No pointers / no REFERENCE TO	Compliant	ADR() confined to SP_Socket and SP_FileIo firmware calls only; documented policy
No ACTIONs	Compliant	All logic in FB bodies and methods; no ACTION blocks anywhere
No arrays of FBs	Compliant	Explicit instances (rob1..rob4, feed1..feed8, ...) + CASE dispatch in PRG_SP_Main
No recursion	Compliant	Method call graph is acyclic; no self or mutual recursion
Methods: VAR/VAR_IN only	Compliant	All methods declare only VAR/VAR_INPUT; results via return value + public out_ members
THIS.Method() from parent	Compliant	All internal calls use THIS., e.g. THIS.Fsm(), THIS.Publish()
No VAR_IN_OUT in user methods	Compliant	VAR_IN_OUT exists only on FW adapter FBs and is used only in FB/program bodies, always assigned

Constraint	Status	Evidence
VAR_IN_OUT always assigned	Compliant	ioAxis/ioGrp bound at every call site in PRG_SP_Main
Constants scope rule (DDT)	Compliant	Limits as GVL VAR CONSTANT; enum access DUT-qualified ET_x#member
PLCopen motion naming	Compliant	MC_* / Group* FBs in adapters; PLCopen Part 1/4 semantics
Comments in English	Compliant	All headers and functional comments in English
Every variable documented	Compliant	All 1,147 declarations carry an explanatory comment; scanner-verified with zero missing (scripts/sp_comment_audit.py)

Table 11-1 - Compliance matrix against the project constraints.

11.2 Open items closure matrix

Item	Status	Resolution
FW-1 full-speed tracking	Worksheet	Firmware/FW_PATCH_ConveyorTracking_IT-EN.md: FB identification in the library repository, SmartPick-to-FW parameter mapping, drop-in variant A (native tracking FB) and B (current cut-off fallback), 5-test commissioning plan; single CONFIRM-TRACKING call site in SP_RobMotion
FW-2 group status	Closed in code	GVL_SP_Fw proxy/inject modes; SP_RobMotion.bFwReadyIn/bFwRefIn inputs; SP_Robot.bFwReady/bFwRef passthrough; PRG_SP_Main wiring from g_abFwGrpReady/g_abFwGrpRef
HW-3 physical I/O map	Template delivered	Hardware/HW_IO_Map_IT-EN.md: safety, 16+16 tool channels, 8 drives, 8 tray sensors, 8 encoder axes, 4 groups, camera triggers, 4 ms task, 9-step checklist
HW-4 camera presets	Closed in code	FUN_SP_CamPreset fill-if-empty family defaults selected via GVL_SP_Hmi.g_aiCamModel; composes with driver protocol presets; CONFIRM tags for on-site job formats
R-05 robot anti-collision	Base closed	SP_ZoneGuard one-latch-per-zone interlock (default OFF, zero behavior change until enabled, deadlock-free)
R-05b advanced crash prevention	Closed in code (v1.2)	SP_Collision: ordered 2-zone acquisition (hierarchy, deadlock-free), per-zone speed limits, dynamic TCP separation with linear slow ramp and latched stop, priority service, wait/contention diagnostics, alarms 30..33; enable with g_stSysCfg.bColAdv
R-07 advanced multi-grip	Closed in code (v1.2)	SP_GripPlanner + SP_ItemQueue.ReserveCluster + SP_ToolCtl.GripSync + robot GripEval state: geometric cluster formation, one synchronized descent, per-cup verification with individual re-grip, payload/CoG audit, synchronized place with async fallback
R-16 professional layer	Closed in code (v1.2)	SP_Oee (OEE = A x P x Q, MTBF/MTTR), SP_Maint (grip/camera/belt/pump wear counters, alarm 40), SP_SelfTest (7 configuration checks, alarm 41); HMI mirrors in GVL_SP_Hmi
R-06 commissioning data	Closed in code	FUN_SP_RobCfgDefault one-shot boot profile applied before SP_Robot.Configure (idempotent); recipe format keeps machine-independent data only

Table 11-2 - Open point closure. FW-1 remains the only engineering activity requiring vendor information.

- **On-site confirmations before production**: firmware tracking FB names (FW-1), group Ready/referenced bit paths in inject mode (FW-2), terminal-level I/O map adaption (HW-3), camera job output formats and ports (HW-4), conveyor encoder scaling per belt, TCP addresses of the cameras, physical calibration of each camera against its feed with SP_Calib2D, velocity-override FB of the iC9200 motion library for the R-05b live speed scaling (CONFIRM in the SP_Robot body).

With the open firmware items resolved, the delivered package compiles on the target, passes the simulation acceptance gate, and is ready for on-machine commissioning following chapter 9. The adapter isolation ensures that any library revision from Yaskawa only affects the five adapter worksheets, keeping the domain logic - the core value of SmartPick_iC - stable across firmware updates.

12. v1.2 - Advanced Multi-Grip & Crash Prevention

12.1 Advanced multi-grip (R-07)

The base library already executed multi-item jobs, but cup by cup: one descent per item, one grip per descent. The advanced multi-grip adds the missing industrial capability: when several products on the belt are geometrically compatible with the cup layout of the gripper head, they are picked with ONE synchronized tracked descent and placed with one synchronized descent as well, multiplying the effective cycle rate without touching the firmware boundary. The intelligence is split into three cooperating pieces. SP_GripPlanner (module 13) is a pure decision engine with a pull API: the queue opens a session on the anchor item (the front candidate), offers every other tracked candidate front-to-rear, and the planner assigns each candidate to the best-matching free cup only when the position matches the cup grid within rMatchTol and the product rules (single type) and payload rules (capacity, minimum gap) are respected. SP_ItemQueue.ReserveCluster commits the accepted members in one atomic reservation and publishes the synchronized TCP pose: because all members ride the same belt, their relative geometry at the pick event equals the capture-time geometry translated by the anchor travel, so the matching is exact and encoder-independent. SP_ToolCtl.GripSync captures the per-cup vacuum picture at the end of the synchronized grip, and the robot FSM consumes it in the new GripEval state.

Failure handling is per cup, not per cycle: a cup that fails the vacuum verification triggers an individual re-grip descent on its own item while the other cups keep holding their parts; when the per-cup retry budget is exhausted the item is flagged NG and the cluster closes normally. The place side mirrors the logic: SP_Robot fits the target formation against the cup layout with the same least-squares math (FUN_SP_FitYaw / FUN_SP_ClusterResidual), performs one synchronized release when the residual is within tolerance and re-places failed cups individually; every degradation path falls back transparently to the classical sequential behavior, so enabling the feature can never make the cell worse. A center-of-gravity audit (rCapCog) degrades an unstable cluster to a safe single pick. The feature is gated per robot by eGripMode = Sync AND uiMaxGrip > 1 AND rMatchTol > 0 and is fully exercised in simulation (TC-09 extended).

12.2 Advanced crash prevention (R-05b)

SP_Collision (module 14) is a superset of the base zone guard and is selected with g_stSysCfg.bColAdv. Each robot requests up to two zones per cycle - the zone of its pick feed and the zone of its place feed. The body normalizes every request pair (clamp, deduplicate, ascending sort) and serves zones strictly in ascending order with a resource-hierarchy rule: a zone is granted only to a robot that holds no zone with a higher id and has no lower own request still pending, which makes circular wait impossible and the guard deadlock-free by construction. Holds are latched until the request drops, so a robot is never disarmed mid-path; free zones are served to the waiter with the best priority rank (ties by robot id). Beyond exclusion, zones can cap the holder velocity scale (transfer and crossing areas), and the dynamic TCP separation monitors every pair of robots that share at least one zone (or all pairs with bSepAll): on approach below rSlowDist the lower-priority robot is slowed along a linear ramp, below rStopDist it is frozen with a latched stop released only with a 10 percent margin against chatter. The velocity scale reaches the motion adapter live through the new rVelScale input of SP_Robot; on the iC9200 target prefer the group velocity override FB at the marked CONFIRM point. Every zone publishes holder, waiters, grant and contention counters, every robot accumulates its frozen time - a direct layout-optimization KPI - and separation stops raise per-robot alarms (ids 30..33) cleared automatically on recovery.

12.3 Professional layer (R-16)

Three compact FBs complete the professional profile of the suite. SP_Oee computes the classical OEE composition - availability over the planned shift time, performance against the ideal cycle time (rIdealCycleMs, HMI editable) and quality from the pick statistics - and samples MTBF/MTTR with an exponential moving average on every fault edge. SP_Maint delta-scans the lifetime counters of the robots and cameras and integrates belt and pump run time into minutes, comparing each wear volume against a configurable threshold with published load percentages and due flags (alarm 40 on the aggregate rising edge, per-family reset commands for the HMI). SP_SelfTest runs seven configuration consistency checks on demand - robot records, multi-grip integrity, outfeed map, feed windows, zone maps, camera addresses and batch definitions - and publishes a structured result (ST_SP_TestRes) with the first failure message (alarm 41 when failing). All three publish plain data mirrors in GVL_SP_Hmi, ready for any HMI generator without additional logic.

New POU / feature	Module	Enable / access
SP_GripPlanner + ReserveCluster (cluster formation)	13 / 05_Tracking	Automatic when the dispatcher sees Sync grip rules in the robot cfg
GripSync + FbkCup + CaptureCups (per-cup vacuum)	08_Robot / SP_ToolCtl	Sync jobs; per-cup channels optional in ST_SP_ToolCfg
GripEval state + individual re-grip	08_Robot	Automatic on failed cups (per-cup retry budget)
Synchronized place + async fallback	08_Robot	Automatic when the target formation fits rMatchTol
SP_Collision (zones + speed + separation)	14_Collision	g_stSysCfg.bColAdv := TRUE + g_aiFeedZone map

New POU / feature	Module	Enable / access
rVelScale live velocity scaling	08_Robot -> SP_RobMotion	Driven by SP_Collision; CONFIRM override FB on target
SP_Oee / SP_Maint / SP_SelfTest	15_Professional	Always active; HMI mirrors + alarms 40/41; shift and maintenance reset commands

Table 12-1 - v1.2 feature map with activation paths.

Behavioral parity is protected by design: with bColAdv = FALSE and the multi-grip rules not matched, the cell behaves exactly like v1.1 - the legacy zone guard remains available, the dispatcher falls back to sequential cup reservations and the robot FSM keeps its classical path. The Python behavioral mirrors (scripts/sp_adv_test.py, 27 scenarios) and the structural QA (66 files) both PASS, extending the offline verification chain of chapter 10 with the cluster, collision and KPI logic.